

CHƯƠNG TRÌNH ĐÀO TẠO **NEXTJS - 2**

HRM.L&D. 2026

1

Sử dụng App Router trong NextJS

2

Phân biệt Client Component & Server Component

3

Tạo form & Validation form trong NextJS

4

Static File Serving trong NextJS

5

Thực hành luyện tập

NextJS App Router

- Next.js từ phiên bản 13 trở lên sử dụng app router, trong đó cấu trúc của thư mục trong app sẽ xác định các routes của ứng dụng. Hệ thống này được gọi là App Router trong Next.js.
- Đầu tiên, ta cần tạo project như sau:

```
>> npx create-next-app@latest
```

- Sau đó, ta thiết lập cho app như sau (có sử dụng App Router)

```
PS C:\Users\Tutorialspoint\Desktop\Articles\Next Js> npx create-next-app@latest
Need to install the following packages:
create-next-app@15.0.3
Ok to proceed? (y) y

✓ What is your project named? ... next-js-example
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to use Turbopack for next dev? ... No / Yes
✓ Would you like to customize the import alias (@/* by default)? ... No / Yes
Creating a new Next.js app in C:\Users\Tutorialspoint\Desktop\Articles\Next Js\
next-js-example.
```

NextJS App Router

- Trong Next.js, mọi file nằm bên trong thư mục app/ sẽ tự động trở thành route. VD, nếu ta tạo một file có tên là app/about/page.tsx, thì file đó sẽ trở thành /about route.
- File app/page.tsx là default route, nếu ta không chỉ định một route thì file đó sẽ trở thành default route.
- Ví dụ:

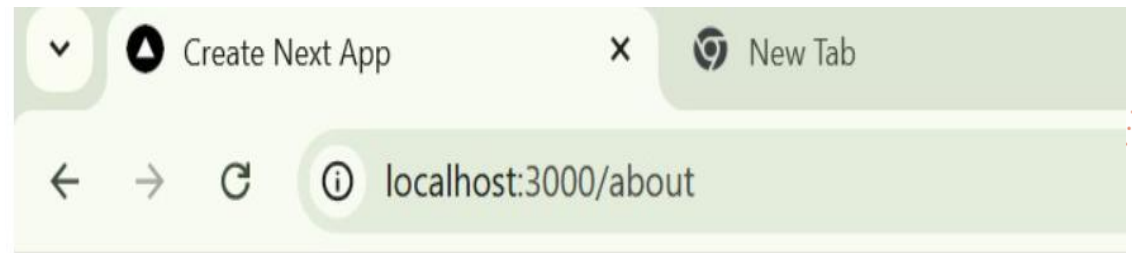
```
my-next-app/  
  app/  
    page.tsx      '/'  
    about/  
      page.tsx    '/about'  
    contact/  
      page.tsx    '/contact'
```

NextJS App Router

- Thiết lập Nested Routes trong Next.js
 - Nested routing là quá trình ta định nghĩa các routes nằm bên trong các routes khác, để tạo nên một cấu trúc phân cấp các routes bên trong ứng dụng web.
 - Ở đây ta tạo ra một thư mục mới với tên là 'about' nằm bên trong thư mục /app/ và ta tạo một file với tên là page.tsx bên trong thư mục đó.
 - Cú pháp và kết quả như sau

```
// File: app/about/page.tsx

export default function About() {
  return (
    <div>
      <h1>About Us</h1>
      <p>Welcome to the about page!</p>
    </div>
  );
}
```



About Us

Welcome to the about page!

NextJS App Router

- Thiết lập dynamic routes
 - Dynamic routing là quá trình định nghĩa các routes có thể chấp nhận các tham số, để người dùng có thể truyền các tham số vào các routes.
 - Ví dụ

```
// File: app/users/[id].tsx

import { useRouter } from 'next/router';

export default function UserProfile() {
  const router = useRouter();
  const { id } = router.query;

  return <div><h1>User Profile: {id}</h1></div>;
}
```

NextJS App Router

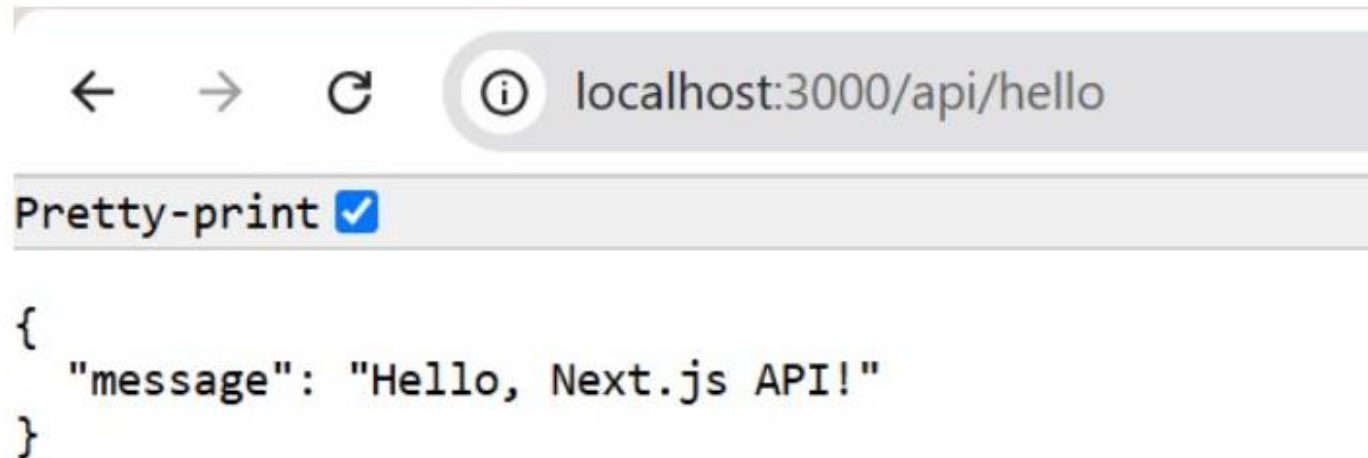
- API Routes
 - API Routes được dùng để tạo ra các API endpoints đơn giản bên trong ứng dụng Next.js. Các API Routes được định nghĩa bên trong thư mục app/api
 - VD, ta có thể tạo một route nhận một POST request đến /api/hello và trả về một JSON response.

```
// File: app/api/hello.js

export default function handler(req, res) {
  res.status(200).json({ message: 'Hello, Next.js API!' });
}
```

NextJS App Router

- Sau đó, khi ta truy cập đến url '/api/hello', ta sẽ thấy kết quả như sau



```
localhost:3000/api/hello

Pretty-print ☒

{
  "message": "Hello, Next.js API!"
}
```


NextJS App Router

- Ví dụ:

```
// File: app/products/[id].tsx

"use client"
import { useParams } from 'next/navigation';

export default function ProductPage() {
  const { id } = useParams();

  return (
    <div>
      <h1>Product {id}</h1>
      <p>This is the product page for item {id}</p>
    </div>
  );
}
```



Header Element

Header will remain here, when user is navigating

Product 123

This is the product page for item 123.

Server Component&Client Component

- Server Component
 - Là component thực thi trên server
 - Render sẵn HTML và không gửi JS xuống trình duyệt
 - Không có tương tác trực tiếp
- Ví dụ

```
// app/posts/page.tsx
export default async function PostsPage() {
  const posts = await fetch("https://api.example.com/posts").then(res => res.json())

  return (
    <ul>
      {posts.map(p => <li key={p.id}>{p.title}</li>)}
    </ul>
  )
}
```

Server Component&Client Component

- Ưu điểm của Server Component
 - Thời gian load nhanh
 - Bảo mật (ẩn API key)
 - Cho phép truy nhập trực tiếp database
 - Giảm JS bundle
- Nhược điểm của Server Component
 - Không sử dụng state
 - Không xử lý event (VD: onClick)
 - Không dùng browser API

Server Component&Client Component

- Client Component
 - Thực thi trên trình duyệt
 - Có JS bundle
 - Có hỗ trợ xử lý tương tác UI
- Ví dụ

```
tsx

"use client"

import { useState } from "react"

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  )
}
```

Server Component&Client Component

- Ưu điểm
 - Hỗ trợ tương tác với người dùng
 - Hỗ trợ hook
 - Hỗ trợ sử dụng browser API
- Hạn chế
 - Tốn JS
 - Thời gian load chậm
 - Không truy cập database trực tiếp
- Chú ý:
 - Server Component có thể import Client Component
 - Client Component không thể import Server Component

Server Component&Client Component

- Ta nên dùng Server Component trong trường hợp:
 - Xử lý fetch data
 - Khi cần render list
 - Khi tiến hành SEO web
 - Khi ta tạo các trang blog, landing
 - Khi ta truy cập đến Database
- Ta không dùng Server Component khi:
 - Có tương tác
 - Khi xử lý animation
 - Khi xử lý form input realtime

Server Component & Client Component

- Ta nên dùng Client Component trong trường hợp:
 - Làm việc với các phần tử form như: Modal, Button
 - Khi cần làm việc với form và validation form
 - Khi cần làm việc với các thành phần giao diện
- Ta không dùng Client Component khi:
 - Không có tương tác
 - Khi chỉ cần hiển thị dữ liệu

Form trong NextJS

- Next.js cung cấp 3 cơ chế để xử lý form
 - Client Form
 - Xử lý tại Client Component
 - Sử dụng khi cần tương tác nhiều
 - Server Action
 - Xử lý tại Server
 - Sử dụng khi ta chỉ cần form đơn giản và SEO
 - API Route
 - Xử lý tại Backend
 - Sử dụng khi ta cần cài đặt logic phức tạp

Form trong NextJS

- Ví dụ form với Client Component

```
"use client"

import { useState } from "react"

export default function LoginForm() {
  const [email, setEmail] = useState("")
  const [error, setError] = useState("")

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault()

    if (!email.includes("@")) {
      setError("Email không hợp lệ")
      return
    }

    console.log(email)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      {error && <p>{error}</p>}
      <button>Submit</button>
    </form>
  )
}
```

Form trong NextJS

- Server Action
 - Là hàm thực thi trên server
 - Cho phép gắn trực tiếp vào form
 - Không cần API route
- Ưu điểm:
 - Giảm bớt mã JS & Code gọn hơn
 - Cho phép SEO tốt hơn
 - Bảo mật hơn
- Ví dụ form với Server Action

```
// app/actions.ts
"use server"

export async function submitForm(formData: FormData) {
  const email = formData.get("email")

  if (!email || !email.toString().includes("@")) {
    throw new Error("Email không hợp lệ")
  }
}
```

```
tsx

// app/page.tsx (Server Component)
import { submitForm } from "../actions"

export default function Page() {
  return (
    <form action={submitForm}>
      <input name="email" />
      <button>Submit</button>
    </form>
  )
}
```

Form trong NextJS

- API Route cho phép viết backend API ngay trong project Next.js, không cần Express/Nest riêng
- Đặc điểm của API Route
 - Thực thi trên server
 - Cho phép truy cập DB, secret key
 - Trả về JSON
 - Sử dụng cho client, mobile app, bên thứ 3 gọi
- Trong App Router (Next.js 13+), API Route được gọi là Route Handler
- Cấu trúc của API Route

```
app/api/  
└─ auth/  
    └─ login/  
        └─ route.ts
```

Form trong NextJS

- Ví dụ API Route đơn giản

```
// app/api/auth/login/route.ts
import { NextResponse } from "next/server"

export async function POST(request: Request) {
  const body = await request.json()

  return NextResponse.json({
    message: "Login thành công",
    data: body,
  })
}
```

Form trong NextJS

- Quá trình validate form trong Next.js

- Validate phía client

```
if (password.length < 8) {  
  setError("Mật khẩu tối thiểu 8 ký tự")  
}
```

- Validate phía server

```
if (!email || !email.includes("@")) {  
  return { error: "Email không hợp lệ" }  
}
```

- Validate với Zod (Khuyến nghị - cần cài thư viện Zod)

```
import { z } from "zod"  
  
export const loginSchema = z.object({  
  email: z.string().email("Email không hợp lệ"),  
  password: z.string().min(8, "Ít nhất 8 ký tự"),  
})
```

Static file serving trong NextJS

- Static file serving
 - Là đặc điểm quan trọng trong Next.js, cho phép xử lý và đáp ứng nhanh chóng các file static mà không cần xử lý ở phía server, VD như: ảnh, font, text, pdf, csv v.v...
- Static file là những tập tin
 - Không thay đổi theo request
 - Không cần xử lý phía backend
 - Được gửi thẳng về cho trình duyệt
- Cách sử dụng
 - Đưa các static file vào thư mục public
 - Sau đó truy cập trực tiếp thông qua root URL

Static file serving trong NextJS

- Ví dụ:

```
public/  
├ images/  
│   └ logo.png  
├ pdf/  
│   └ guide.pdf  
├ favicon.ico  
└ robots.txt
```

- Cách truy cập:

```
  
<a href="/pdf/guide.pdf">Download</a>
```

- Những file không nên đặt trong thư mục public
 - File cần bảo mật
 - File phụ thuộc theo user
 - File lớn theo session

NextJS Lab

- Thực hành luyện tập với NextJS

THANKS FOR WATCHING



4th floor – Hoa Binh tower, 106 Hoang Quoc Viet, Hanoi, Vietnam



(+84) 366 496 399



sales.en@luvina.net



www.luvina.net